

Computer Vision Lab-2

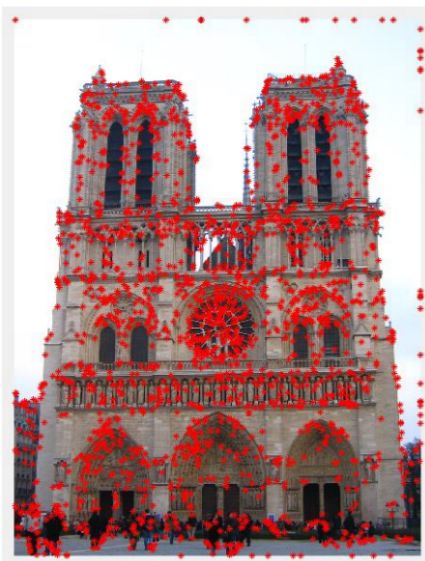
This lab has three parts. We will use Python environment throughout. Part one implements a version of the Harris keypoint detector. Part two implements a simple descriptor matching method and uses it for object matching. Part three implements a simple RANSAC routine for robust matching.

Part 1. Harris Detector Implementation

Try to program a simple implementation of the Harris corner detector. You can use the provided `corner_detection_template.py` as a template if you wish. Load an image, compute the Harris keypoints and plot them on the image. You will first get the requisite gradients using convolution as studied in the lecture. Then compute the Harris matrix at each point, the Harris score at each point, and threshold this score to obtain the keypoint detections. A summary of required steps is given by Algorithm 4.1 of the Szeliski book. This is reprinted below, but see the lecture notes and Szeliski book Ch 4.1.1 for details. For simplicity you can compute the Harris score at each point pointwise rather than summing over a local window.

1. Compute the horizontal and vertical derivatives of the image I_x and I_y by convolving the original image with derivatives of Gaussians (Section 3.2.3).
2. Compute the three images corresponding to the outer products of these gradients. (The matrix A is symmetric, so only three entries are needed.)
3. Convolve each of these images with a larger Gaussian.
4. Compute a scalar interest measure using one of the formulas discussed above.
5. Find local maxima above a certain threshold and report them as detected feature point locations.

The result should look something like this:



You can compare your results with using predefined opencv function `cv2.cornerHarris()`.

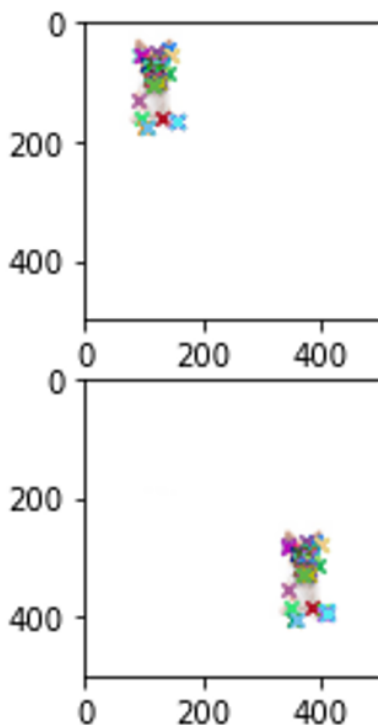
Bonus [Moderate]: The naïve way to implement the Harris score computation involves looping over all pixels which is slow. Figure out how to vectorise the algorithm and eliminate the for loop entirely.

Part 2: Descriptor matching

In this section, we'll implement simple descriptor matching. For interest point detection, now use `cv2.cornerHarris()` as it is likely to be more robust than your own implementation. Examine the provided template `descriptor_matching_template.py`. The initial script creates two images to match by taking an input image and its translated version. Your goal is: (1) to write the routines `extract_patch_descriptors()` for extracting descriptors and (2) `match_descriptors()` for matching them. Assuming N keypoints have been detected, then:

- `extract_patch_descriptors()` should take the $N \times 2$ list of detected keypoint coordinates, and return an $N \times \text{PatchSize}$ matrix, where each row encodes the vector of pixels centred at each patch.
- `match_descriptors()` should take one descriptor matrix for each image, and return the $N \times 1$ vector of matches. Each element of this vector says for the corresponding keypoint in image1, which is the corresponding keypoint in image2. No matches can be encoded by a negative number.

The result should look something like the below, where the first row shows the initial detected interest points and the second row shows the result of matching them. The matches are color coded, and you can see the corresponding points in each image by matching colors.



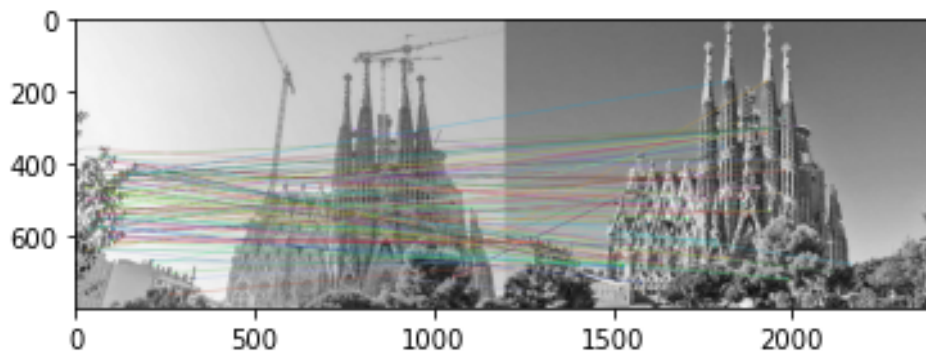
Bonus [Moderate]: Rather than using the initial images `ka.jpg` and its translated version `kb.jpg`, try to load up `1.jpg` and `2.jpg` as the first and second images. Try to implement a

gradient-based descriptor and matching routine that is robust enough to match points between these two.

Part 3: Image alignment with RANSAC

In this section, we'll use RANSAC to do robust estimation of an affine image transformation based on pairs of matched interest points. Examine the provided script `image_alignment_template.py`. It is expected to load two images and match features between them.

Your task is to write a RANSAC loop that robustly computes the affine transformation between the left and right image. As per the given example, it should return the 3×1 transformation M that takes homogeneous points X_1 in image 1 and transforms them to match their corresponding points X_2 in image 2. IE: $X_2 = MX_1$. But to do this robustly with RANSAC, your routine should repeatedly sample different subsets of matched points to compute the transformation, and then score them by the number of inliers. Return the transformation with the highest inlier score.



Bonus [Moderate]: Use the RANSAC loop to compute alignments, and average the two aligned images together to make a panorama.